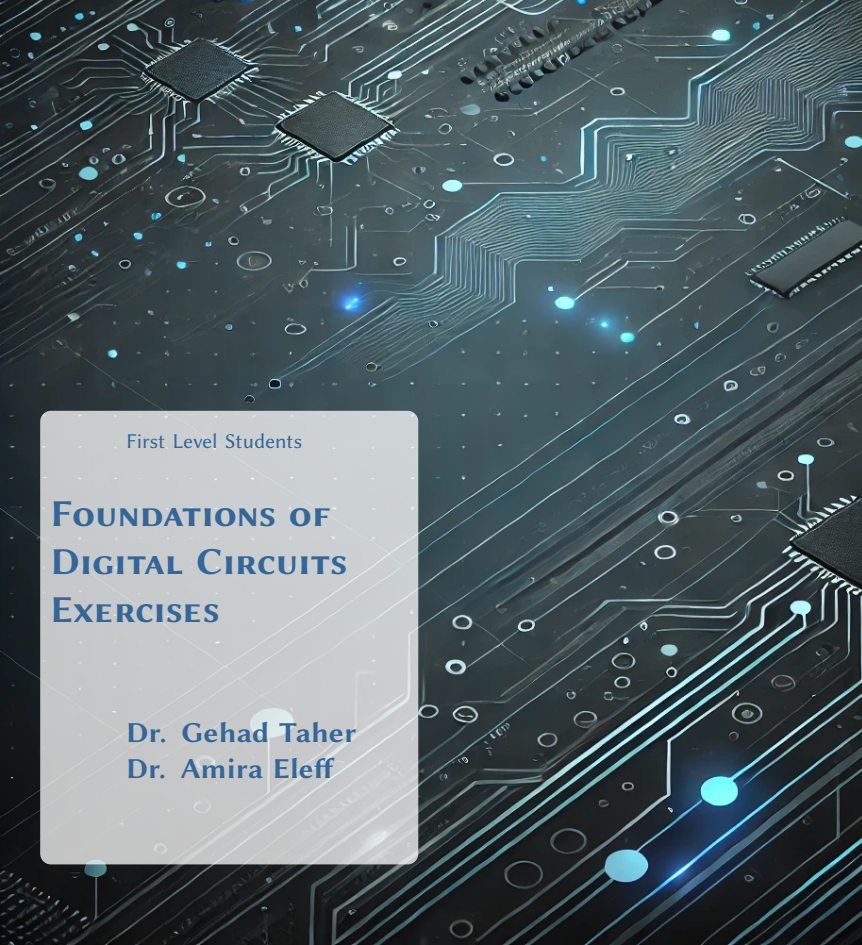




First Level Students

FOUNDATIONS OF DIGITAL CIRCUITS EXERCISES

Dr. Gehad Taher
Dr. Amira Eleff

Contents

Contents	2
1 Introduction to Digital Systems Lab & Exercises	4
1.1 Lab 1: Introduction to Digital Systems Basics . . .	4
2 Binary Arithmetic and Number Systems	12
2.1 Exercises	18
3 Logic Gates Lab Work and Exercises	29
3.1 Homework Questions	34
3.2 Exercises	35

4	Boolean Algebra and Simplifications	42
5	Combinational Circuits	43
6	Sequential Circuits	44

Chapter 1

Introduction to Digital Systems Lab & Exercises

1.1 Lab 1: Introduction to Digital Systems Basics

Objective

Learn to simulate analog-to-digital conversion (ADC) and visualize binary outputs using Antares simulation.

Detailed Steps in Antares Simulation

1. Open Antares Simulation Tool

1. Launch Antares on your computer.
2. Click on **New Project** to start a fresh workspace.
3. Name your project: *Lab 1 - Digital Systems Basics*.
4. Choose the appropriate grid size for easy placement of components.

2. Place an Analog Signal Source

1. Go to the **Components Toolbar** (usually on the left side).
2. Navigate to *Signal Sources* → *Analog Sources* → Select **Sine Wave Generator**.
3. Drag the sine wave generator into the workspace.
4. **Configure the Sine Wave Generator:**
 - Double-click the sine wave component to open its properties.

- Set the following parameters:
 - **Amplitude:** 5 V
 - **Frequency:** 1 kHz
 - **Offset Voltage:** 0 V
 - **Phase:** 0°
- Save the configuration and ensure the waveform icon appears on the generator.

3. *Add an 8-bit ADC Block*

1. From the **Components Toolbar**, go to *Conversion Devices* → **ADC (Analog-to-Digital Converter)**.
2. Drag an **8-bit ADC** into the workspace.
3. **Connect the Input:**
 - Use the wire tool to connect the sine wave generator's output to the input pin of the ADC.

4. *Add Binary Output Visualization*

1. Navigate to *Display Components* → **LED Indicators**.

2. Place **8 LED Indicators** in a horizontal row.
3. Connect the **output pins of the ADC** to the **corresponding LEDs**:
 - ADC Output Pin 0 $\rightarrow$ LED 0
 - ADC Output Pin 1 $\rightarrow$ LED 1
 - ... and so on, until Output Pin 7 $\rightarrow$ LED 7.

5. *Add a Decoder and 7-Segment Display*

1. Go to *Display Components* $\rightarrow$ **Decoder + 7-Segment Display**.
2. Place a **Binary to Decimal Decoder** and a **7-Segment Display** in the workspace.
3. Connect the output pins of the ADC to the input pins of the decoder.
4. Connect the decoder output to the 7-segment display.

6. *Simulation Setup*

1. **Set the Simulation Time:**

- Go to the **Simulation Menu** → **Settings** → Set the simulation time to 10 ms.

2. **Run the Simulation:**

- Click the **Run** button to start the simulation.
- Observe the following:
 - **LED Indicators:** Display binary representation of the sine wave.
 - **7-Segment Display:** Displays the decimal value of the sine wave's current digital output.

7. *Experiment with Sampling Rates*

1. **Modify ADC Sampling Rate:**

- Double-click the ADC block to open its properties.
- Adjust the sampling rate to values like 1 kHz, 5 kHz, and 10 kHz.

2. **Run the simulation for each sampling rate.**
3. **Observation:**
 - Note how the LED blinking patterns and decimal output on the 7-segment display change with different sampling rates.

Analysis Questions

1. How does changing the sampling rate affect the accuracy of the digital output?
2. What is the impact of a low sampling rate on the sine wave's digital representation?
3. Explain how the number of ADC bits impacts the resolution of the digital output.

Exercises

Part A: Theoretical Questions

1. Explain the main differences between digital and analog systems in terms of data representation, precision, and noise resistance. Provide one example for each.

2. Compare digital media (e.g., MP3, MP4) with analog media (e.g., cassette tapes, VHS). Highlight the role of digital systems in improving media quality and accessibility.
3. List three applications of digital systems in daily life and explain how digital systems enhance their functionality.
4. How do digital systems store large amounts of data efficiently? Discuss the role of binary representation in storage systems like hard drives and SSDs.
5. Why is hexadecimal used in digital systems, especially in programming and memory addressing? Provide one practical example.
6. How does the discrete nature of digital signals make them more immune to noise compared to analog signals? Use a communication system as an example.
7. What makes binary representation particularly well-suited for digital systems? Discuss its relationship with transistor-based circuits.

8. Calculate the duty cycle of a waveform with $t_W = 5$ ms and $T = 20$ ms.
9. Draw a timing diagram for the following binary sequence: 110101.
10. Define the concept of sampling rate in digital systems and explain its impact on the quality of digitized signals.
11. What are the advantages of digital error detection and correction in communication systems?
12. Discuss the role of Boolean algebra in designing digital circuits and provide one practical example.
13. Why are digital systems often preferred for automation in industries? Give three examples of industrial applications.
14. Explain how analog-to-digital conversion (ADC) works and its importance in bridging analog and digital systems.
15. How do flip-flops enable digital memory in storage registers? Discuss one example of their use.

Chapter 2

Binary Arithmetic and Number Systems

Lab Work for Binary Arithmetic and Number Systems

Number System Conversions

Example 1: Convert 27_{10} to binary.

1. Divide the decimal number by 2 and record the remainder:

$$27 \div 2 = 13 \text{ remainder: } 1, \quad 13 \div 2 = 6 \text{ remainder: } 1,$$

$$6 \div 2 = 3 \text{ remainder: } 0, \quad 3 \div 2 = 1 \text{ remainder: } 1, \quad 1 \div 2 = 0 \text{ remainder: } 1$$

2. Read the remainders from bottom to top:

$$27_{10} = 11011_2.$$

Example 2: Convert 101101_2 to decimal.

1. Write the binary number with positional weights:

$$101101_2 = (1 \cdot 2^5) + (0 \cdot 2^4) + (1 \cdot 2^3) + (1 \cdot 2^2) + (0 \cdot 2^1) + (1 \cdot 2^0).$$

2. Calculate the value:

$$32 + 0 + 8 + 4 + 0 + 1 = 45.$$

3. Result:

$$101101_2 = 45_{10}.$$

Example 3: Convert 456_{10} to octal.

1. Divide the decimal number by 8 and record the remainders:

$$456 \div 8 = 57 \text{ remainder: } 0, \quad 57 \div 8 = 7 \text{ remainder: } 1, \quad 7 \div 8 = 0 \text{ remainder: } 7$$

2. Read the remainders from bottom to top:

$$456_{10} = 710_8.$$

Example 4: Convert $3A_{16}$ to binary.

1. Convert each hexadecimal digit to 4-bit binary:

$$3 = 0011, \quad A = 1010.$$

2. Combine the binary equivalents:

$$3A_{16} = 00111010_2.$$

Binary Arithmetic Operations

Example 5: Add $1011_2 + 1101_2$.

1. Align the numbers and add bit by bit:

$$\begin{array}{r}
 1011 \\
 +1101 \\
 \hline
 11000 \\
 \begin{array}{l} 14 \quad | \quad 44 \end{array}
 \end{array}$$

2. Result:

$$1011_2 + 1101_2 = 11000_2.$$

Example 6: Subtract $1100_2 - 1011_2$ using two's complement.

1. Find the two's complement of 1011_2 :

- Invert all bits: 0100_2 .
- Add 1: $0100_2 + 1 = 0101_2$.

2. Add $1100_2 + 0101_2$:

$$\begin{array}{r} 1100 \\ +0101 \\ \hline 10001 \end{array}$$

Discard the carry.

3. Result:

$$1100_2 - 1011_2 = 0001_2.$$

Error Detection and Correction

Example 7: Add Even Parity to 1010110_2 .

1. Count the number of 1s in the data:

$$1 + 0 + 1 + 0 + 1 + 1 + 0 = 4.$$

2. Since the number of 1s is even, the parity bit is 0 (for even parity).
3. Append the parity bit to the data:

$$1010110_2 \rightarrow 10101100_2.$$

Example 8: Detect an Error in 10101110_2 .

1. Count the number of 1s in the received data:

$$1 + 0 + 1 + 0 + 1 + 1 + 1 + 0 = 5.$$

2. Since the number of 1s is odd and the parity bit is set for even parity, this indicates an error in transmission.

Homework Questions

Number System Conversions

1. Convert the following decimal numbers to binary:

- 19_{10}
- 85_{10}

2. Convert the following binary numbers to decimal:

- 11101_2
- 100011_2

3. Convert 573_{10} to octal and hexadecimal.

Binary Arithmetic Operations

4. Perform the following binary additions:

- $1001_2 + 1101_2$
- $1110_2 + 1011_2$

5. Subtract $10010_2 - 1011_2$ using two's complement.

Error Detection and Correction

6. Add even parity to the following data words:

- 1101010_2
- 1011101_2

7. Detect and correct an error in the following received data assuming even parity:

- 11010101_2
- 10101110_2

2.1 Exercises

Do the following conversion problems

1. Convert the following decimal values into Binary, Octal, and Hexadecimal:
 - a) $(25)_{10}$
 - b) $(63)_{10}$
 - c) $(125)_{10}$

- d) $(50)_{10}$
- e) $(255)_{10}$
- f) $(45)_{10}$
- g) $(90)_{10}$
- h) $(150)_{10}$
- i) $(75)_{10}$
- j) $(300)_{10}$
- k) $(10)_{10}$
- l) $(200)_{10}$
- m) $(500)_{10}$
- n) $(1000)_{10}$
- o) $(2048)_{10}$

2. Converting Fractional Decimal Numbers:

Part A: Convert to Binary

Convert the following fractional decimal numbers into binary:

- a) 5.25

- b) 12.625
- c) 0.40625
- d) 3.1416 (to 4 binary places)
- e) 18.375
- f) 7.5
- g) 9.125
- h) 0.6875
- i) 4.9375 (to 4 binary places)
- j) 22.8125
- k) 0.3125
- l) 2.7183 (to 4 binary places)
- m) 50.625
- n) 100.125
- o) 0.0009765625

Part B: Convert to Octal

Convert the following fractional decimal numbers into octal:

- a) 10.125

- b) 45.6875
- c) 0.15625
- d) 7.8125
- e) 32.9375
- f) 15.375
- g) 48.9375
- h) 0.78125
- i) 11.625
- j) 40.15625
- k) 63.125
- l) 5.375
- m) 25.75
- n) 0.984375
- o) 8.0625

Part C: Convert to Hexadecimal

Convert the following fractional decimal numbers into hexadecimal:

- a) 14.375
- b) 0.0625
- c) 25.8125
- d) 50.5
- e) 100.125
- f) 18.25
- g) 0.9375
- h) 33.375
- i) 70.0625
- j) 200.5
- k) 150.0625
- l) 0.15625
- m) 75.25
- n) 0.984375
- o) 6.625

3. Add the following binary numbers:

- a) $10111 + 01101$
- b) $1101 + 1010$
- c) $110 + 100$
- d) $111 + 11$
- e) $111 + 101$
- f) $10101 + 11001$
- g) $1000 + 0111$
- h) $10101 + 00110$
- i) $110011 + 100010$
- j) $10101 + 01010$
- k) $11111 + 00001$
- l) $100000 + 000001$
- m) $101 + 110$
- n) $1111 + 1110$
- o) $110000 + 000111$

4. Perform the following binary subtractions:

- a) $1101 - 0100$
- b) $1001 - 0111$
- c) $10011 - 10010$
- d) $110000 - 1111$
- e) $10101 - 01101$
- f) $1110 - 1001$
- g) $11010 - 10101$
- h) $100100 - 1100$
- i) $111100 - 1010$
- j) $101010 - 100001$
- k) $11001 - 1010$
- l) $1110 - 0011$
- m) $101110 - 100111$
- n) $111111 - 110110$
- o) $1101 - 0110$

5. Perform the following binary multiplications:

- a) $11 * 10$
- b) $101 * 11$
- c) $111 * 110$
- d) $1100 * 101$
- e) $1110 * 1110$
- f) $1001 * 110$
- g) $1011 * 101$
- h) $1101 * 100$
- i) $1111 * 1010$
- j) $1001 * 111$
- k) $101 * 10$
- l) $110 * 11$
- m) $1110 * 1011$
- n) $11101 * 1011$
- o) $100101 * 1111$

6. Divide the binary numbers as indicated:

a) $110 \div 11$

b) $1010 \div 10$

c) $1111 \div 101$

d) $1101 \div 10$

e) $10101 \div 101$

f) $111000 \div 1000$

g) $101100 \div 111$

h) $1001 \div 100$

i) $101101 \div 11$

j) $110111 \div 110$

k) $111111 \div 1111$

l) $101001 \div 110$

m) $100000 \div 100$

n) $101101 \div 11$

o) $11011 \div 10$

7. Obtain the 1's and 2's complements of the following binary numbers:

a) 00010000

b) 11011010

c) 10000101

d) 10101010

e) 11111111

f) 10101010

g) 11100011

h) 10101111

i) 10001110

j) 10100101

k) 01111000

l) 11001100

m) 10010110

n) 11111100

o) 10101001

8. Express each decimal number in binary as an 8-bit sign-magnitude number:

- (a) +29 (b) -85 (c) +100 (d) -123
(e) +56 (f) -75 (g) +42 (h) -100
(i) +78 (j) -90 (k) +127 (l) -200
(m) +15 (n) -32 (o) +99

9. Convert each hexadecimal number to binary:

- (a) $10A4$ (b) $CF8E$ (c) 9742
(d) $ABCD$ (e) $B4$ (f) 54
(g) $1C3$ (h) $2F5$ (i) AA (j) $4E$
(k) $7F$ (l) 333 (m) 120 (n) ABC (o) FA

Chapter 3

Logic Gates Lab Work and Exercises

How to Use Antares

1. **Install Antares:** Download and install Antares from the official website.
2. **Create a New Circuit:** Open Antares and create a new project.
3. **Add Components:** Use the toolbox on the left to drag and

drop input pins, gates, and output pins onto the workspace.

4. **Connect Components:** Use the wiring tool to connect inputs to gates and gates to outputs.
5. **Test the Circuit:** Provide binary inputs (0 or 1) to the input pins and observe the output.

Implementation of Logic Gates in Antares

AND Gate

1. Drag and drop:
 - Two input pins (A and B).
 - One AND gate.
 - One output pin.
2. Connect:
 - Connect A and B to the inputs of the AND gate.
 - Connect the output of the AND gate to the output pin.
3. Test:

- Input: $A = 0, B = 0 \rightarrow$ Output: $Y = 0$.
- Input: $A = 1, B = 1 \rightarrow$ Output: $Y = 1$.

OR Gate

1. Drag and drop:

- Two input pins (A and B).
- One OR gate.
- One output pin.

2. Connect:

- Connect A and B to the inputs of the OR gate.
- Connect the output of the OR gate to the output pin.

3. Test:

- Input: $A = 0, B = 1 \rightarrow$ Output: $Y = 1$.
- Input: $A = 1, B = 0 \rightarrow$ Output: $Y = 1$.

NOT Gate

1. Drag and drop:

- One input pin (A).
- One NOT gate.
- One output pin.

2. Connect:

- Connect A to the input of the NOT gate.
- Connect the output of the NOT gate to the output pin.

3. Test:

- Input: $A = 0 \rightarrow$ Output: $Y = 1$.
- Input: $A = 1 \rightarrow$ Output: $Y = 0$.

NAND Gate

1. Drag and drop:

- Two input pins (A and B).
- One NAND gate.

- One output pin.
2. Connect:
 - Connect A and B to the inputs of the NAND gate.
 - Connect the output of the NAND gate to the output pin.
 3. Test:
 - Input: $A = 1, B = 1 \rightarrow$ Output: $Y = 0$.
 - Input: $A = 0, B = 1 \rightarrow$ Output: $Y = 1$.

XNOR Gate

1. Drag and drop:
 - Two input pins (A and B).
 - One XNOR gate.
 - One output pin.
2. Connect:
 - Connect A and B to the inputs of the XNOR gate.

- Connect the output of the XNOR gate to the output pin.

3. Test:

- Input: $A = 1, B = 1 \rightarrow$ Output: $Y = 1$.
- Input: $A = 1, B = 0 \rightarrow$ Output: $Y = 0$.

3.1 *Homework Questions*

1. Implement a 3-input AND gate in Antares. Test it with all possible input combinations.
2. Design a circuit in Antares to simulate $Y = A \cdot B + \overline{C}$.
3. Create a 3-input OR gate and verify its output for inputs $A = 1, B = 0, C = 1$.
4. Design a NAND gate using only AND and NOT gates in Antares.
5. Implement an XOR gate using only NAND gates in Antares.
6. Construct a 4-input NOR gate and verify its truth table.

7. Simulate a 2-bit binary adder in Antares. Use XOR for sum and AND for carry.
8. Design a circuit to compute $Y = A \oplus B \cdot \overline{C}$.
9. Create a circuit for a 3-bit comparator ($A > B$).
10. Implement a traffic light controller circuit with Red, Yellow, and Green signals.

3.2 Exercises

1. Truth Table and Logic Outputs

- a) Complete the truth table for the expression $F = A \cdot B + \overline{C}$.

A	B	C	F
0	0	0	
0	1	1	
1	0	1	
1	1	0	

- b) Derive the truth table for the XOR operation $F = A \oplus B$.

- c) For $F = A \cdot B + A \cdot \overline{C}$, compute the truth table for all possible inputs.
- d) Complete the truth table for $G = \overline{A \cdot B} \cdot (C + D)$.
- e) Determine the output $H = \overline{A + B \cdot C}$ for all possible inputs.
- f) Complete the truth table for $F = (A + B) \cdot (\overline{C} + D)$.
- g) Derive the truth table for a 4-input AND gate.
- h) Compute the truth table for $F = \overline{A} \cdot \overline{B} + A \cdot \overline{C}$.
- i) Complete the truth table for the full-adder circuit:

$$S = A \oplus B \oplus C, \quad C_{out} = A \cdot B + C \cdot (A \oplus B).$$

- j) Derive the truth table for $F = \overline{A \cdot B} + \overline{C}$.

2. Circuit Design

- a) Draw the circuit for $F = A \cdot B + \overline{C}$.
- b) Design a circuit for $G = \overline{A \cdot B} \cdot C$.
- c) Implement a logic circuit for a 2-bit comparator ($A > B$).

- d) Design a vending machine circuit where:
- Product dispenses when either Coin A and Coin B are inserted, or Coin C alone is inserted.
- e) Create a custom logic gate for $F = \overline{A} \cdot \overline{B} + C$.
- f) Design a traffic light controller circuit where:
- Red light is on when $R = A \cdot \overline{B}$.
 - Green light is on when $G = \overline{A} \cdot B$.
 - Yellow light is on when $Y = A \cdot B$.
- g) Create a circuit to compute $F = (A + B) \cdot \overline{C}$.
- h) Design a circuit for an alarm system that activates when:
- Either motion is detected, or
 - A door/window is open while the system is armed.
- i) Implement a full-adder circuit using XOR and AND gates.
- j) Design a circuit for $F = \overline{A + B} + (C \cdot D)$.
- k) Implement a majority voting circuit for three inputs A, B, C , where $F = 1$ if at least two inputs are 1.

- l) Design a parity generator circuit for four inputs, producing $F = 1$ for even parity.
- m) Create a 3-input XOR circuit and draw its corresponding logic diagram.
- n) Design a logic circuit to control a parking barrier that opens when:
 - A valid card is swiped, or
 - A remote button is pressed by security staff.
- o) Implement a circuit for $F = (A \cdot B) + (\overline{C} \cdot D)$ using only NAND gates.

3. Fault Analysis and Optimization

- a) Examine the following circuit and identify the faulty gates:
 - Input $A = 1, B = 0$, Output $F = 1$ (Expected: 0).
 - Input $A = 0, B = 1$, Output $G = 0$ (Expected: 1).
- b) Suggest replacements for two gates in a circuit that could be replaced by a single universal gate.

- c) Analyze the truth table below and identify optimization opportunities:

A	B	C	F
0	0	0	1
0	1	1	1
1	0	1	0
1	1	0	1

- d) Suggest a more optimized circuit for the expression $F = A \cdot B + A \cdot C$.
- e) Replace a combination of XOR and AND gates with a single NAND gate.
- f) Identify and correct faults in the following logic circuit for:

$$F = A \cdot (B + C).$$

- g) Examine the propagation delay in a faulty circuit and suggest solutions to minimize it.
- h) Propose gate replacements to reduce power consumption in a given circuit.

- i) Identify which gates in a circuit are redundant and can be removed without affecting functionality.
- j) Optimize the following circuit by minimizing the number of logic gates:

$$F = (A + B) \cdot (C + \overline{D}).$$

4. Real-Life Applications

- a) Design a logic circuit to control an elevator system:
 - $F = 1$ if the Floor 1 button is pressed, or the Floor 2 button is pressed while the system is idle.
- b) Create a circuit for a digital clock where $F = 1$ when Hour is 12 and Minute is 00.
- c) Implement a voting system circuit:
 - $F = 1$ when at least two out of three inputs (Yes votes) are true.
- d) Create a logic circuit for a security system that:
 - Activates if Motion is detected or if a window is open and the system is armed.

- e) Design a logic circuit for a parking lot system that:
- Opens the gate if there is an available parking slot and a car is detected.

Chapter 4

Boolean Algebra and Simplifications

Chapter 5

Combinational Circuits

Chapter 6

Sequential Circuits